

Week 13 - CH08. 텍스트 분석

06 토픽 모델링(Topic Modeling) - 20 뉴스 그룹

토픽 모델링

- 문서 집합에 숨어 있는 중요 주제 찾아냄
 - 인간 토픽 모델링은 더 함축적 의미로 문장 요약
 - 머신러닝 토픽 모델링은 숨겨진 주제 효과적으로 표현 가능한 중심 단어를 함축적으로 추출
 - LDA(Latent Dirichlet Allocation)
 - 20 뉴스그룹
 - 20가지의 주제 가진 뉴스그룹 데이터 보유
-
- `fetch_20newsgroups()` API는 `categories` 파라미터를 통해 필요한 주제만 필터링해 추출
 - 추출된 텍스트를 Count 기반 벡터화 변환
 - `max_features = 1000`으로 word 피처의 개수 제한
 - `ngram_range`는 (1,2)로 설정, feature 벡터화 변환

```

from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation

# 모터사이클, 야구, 그래픽스, 윈도우즈, 중동, 기독교, 의학, 우주 주제를 추출.
cats = ['rec.motorcycles', 'rec.sport.baseball', 'comp.graphics', 'comp.windows.x',
        'talk.politics.mideast', 'soc.religion.christian', 'sci.electronics', 'sci.med' ]

# 위에서 cats 변수로 기재된 category만 추출. fetch_20newsgroups( )의 categories에 cats 입력
news_df= fetch_20newsgroups(subset='all', remove=('headers', 'footers', 'quotes'),
                             categories=cats, random_state=0)

# LDA는 Count 기반의 Vectorizer만 적용합니다.
count_vect = CountVectorizer(max_df=0.95, max_features=1000, min_df=2, stop_words='english',
                              feat_vect = count_vect.fit_transform(news_df.data)
print('CountVectorizer Shape:', feat_vect.shape)

CountVectorizer Shape: (7862, 1000)

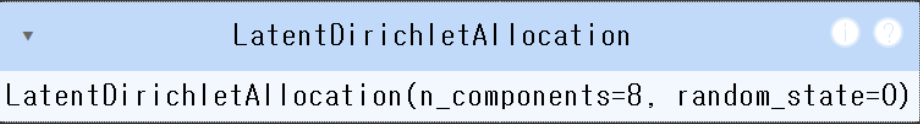
```

- CountVectorizer 객체 변수인 feat_vect 모두 7862개의 문서가 1000개의 feature로 구성된 행렬데이터
- feature vector화된 데이터 세트를 기반으로 LDA 토픽 모델링 수행

```

lda = LatentDirichletAllocation(n_components=8, random_state=0)
lda.fit(feat_vect)

```



- lda.fit()으로 components_ 속성값을 가지게 됨
- components_는 개별 토픽별로 각 word feature가 얼마나 많이 그 토픽에 할당됐는지에 대한 수치
 - 높은 값일수록 해당 word feature는 그 토픽의 중심 word가 됨
- components_의 형태와 속성값을 확인

```
print(lda.components_.shape)
lda.components_

(8, 1000)
array([[2.69030238e+02, 1.87798026e+02, 7.09003824e+01, ...,
        1.22710343e+01, 1.06329639e+02, 7.25995512e+01],
       [1.25091799e-01, 2.46049106e+00, 1.25051902e-01, ...,
        2.80071176e+02, 1.25089783e-01, 5.05669662e+01],
       [1.33978420e+02, 1.25042012e-01, 9.98277256e+01, ...,
        1.25092219e-01, 3.31078261e+01, 1.25028398e-01],
       ...,
       [2.98813886e+01, 1.88071366e+01, 1.14748730e+01, ...,
        1.93022584e+01, 5.29368271e+00, 1.44478198e+01],
       [1.25074899e-01, 1.25105300e-01, 1.25004235e-01, ...,
        1.03576436e+02, 1.25100535e-01, 7.22276359e+01],
       [1.25172284e-01, 1.03967760e+00, 1.25221075e-01, ...,
        5.31740996e+01, 1.25025929e-01, 1.25062991e-01]])
```

- 8개 토픽별로 1000개의 word feature가 해당 토픽별로 연관도 값을 가짐
- components_array의 0번째 row
 - 10번째 col에 있는 값은 Topic #0에 대해 feature 벡터화된 행렬에서 10번째 칼럼에 해당하는 feature가 Topic #0에 연관된 수치 가짐
- 토픽별 word 연관도를 보기는 힘들
- display_topics() 함수를 만들어 각 토픽별 연관도 높은 순으로 Word 나열

```
def display_topics(model, feature_names, no_top_words):
    for topic_index, topic in enumerate(model.components_):
        print('Topic #', topic_index)

        # components_ array에서 가장 값이 큰 순으로 정렬했을 때, 그 값의 array index를 반환.
        topic_word_indexes = topic.argsort()[::-1]
        top_indexes = topic_word_indexes[:no_top_words]

        # top_indexes대상인 index별로 feature_names에 해당하는 word feature 추출 후 join으로
        feature_concat = ' '.join([feature_names[i] for i in top_indexes])
        print(feature_concat)

# CountVectorizer객체내의 전체 word들의 명칭을 get_features_names()를 통해 추출
feature_names = count_vect.get_feature_names_out()

# Topic별 가장 연관도가 높은 word를 15개만 추출
display_topics(lda, feature_names, 15)
```

```
Topic # 0
10 year medical health 1993 20 12 disease cancer team patients research number new 11
Topic # 1
don just like know think good time ve does way really people want ll right
Topic # 2
image file jpeg output program gif images format files color entry use bit 03 02
Topic # 3
armenian armenians turkish people said turkey armenia government genocide turks muslim russiar
Topic # 4
israel jews dos jewish israeli dos dos arab state people arabs palestinian adl ed anti peace
Topic # 5
edu com available graphics ftp window use mail data motif software version pub information sei
Topic # 6
god people jesus church believe say christ does christian think christians did know bible man
Topic # 7
thanks use using does help like display need problem know server screen windows window prograr
```

- 모터사이클, 야구, 그래픽스, 윈도우즈, 중동, 기독교, 전자공학, 의학 추출
- Topic #0
 - 일부 불분명한 주제어 있음
 - 주로 의학에 관련된 주제어 추출
- Topic #1
 - 명확하지 않음
 - 일반적 단어가 주를 이룸
- Topic #2
 - 컴퓨터 그래픽스 영역의 주제어가 다수 포함
- Topic #3
 - 일반적 단어로 주제어 추출
- Topic #4

- 명확하게 중동 지역의 주제어 추출
- Topic #5
 - 일부 컴퓨터 그래픽스 영역의 주제어
 - 전반적인 컴퓨터 용어라 토픽 정하기는 어려움
- Topic #6
 - 명확한 기독교 관련 주제어
- Topic #7
 - 윈도우 운영 체제와 관련된 주제어 추출

07 문서 군집화 소개와 실습(Opinion Review 데이터 세트)

문서 군집화 개념

- 비슷한 텍스트 구성의 문서를 군집화
- 동일한 군집에 속하는 문서를 같은 카테고리 소속으로 분류
- 텍스트 분류 기반의 문서 분류와 유사
- 문서 분류는 사전에 결정 카테고리 값을 가진 학습 데이터 세트 필요
- 문서 군집화는 학습 데이터 세트가 필요 없는 비지도 학습 기반으로 동작

Opinion Review 데이터를 이용한 문서 군집화 수행

```

import glob, os
import warnings
warnings.filterwarnings('ignore')
pd.set_option('display.max_colwidth', 700)

# 업로드한 파일들은 모두 /content에 저장됨
path = '/content'
all_files = glob.glob(os.path.join(path, "*.data"))
filename_list = []
opinion_text = []

for file_ in all_files:
    # 파일을 table 형식으로 읽을 수 있을 때
    try:
        df = pd.read_table(file_, index_col=None, header=0, encoding='latin1')
        content = df.to_string()
    except:
        # 파일이 표 형식이 아니면 그냥 텍스트로 읽기
        with open(file_, encoding='latin1') as f:
            content = f.read()
        filename_ = file_.split('/')[-1]
        filename = filename_.split('.')[0]
        filename_list.append(filename)
        opinion_text.append(content)

document_df = pd.DataFrame({'filename': filename_list, 'opinion_text': opinion_text})
print(document_df.shape)
document_df.head()

```

(102, 2)

	filename	opinion_text
0	interior_toyota_camry_2007	First of all, the interior has way too many cheap plastic parts like the cheap plastic center piece that houses the clock .\n0 3 blown struts at 30,000 miles, interior trim coming loose and rattling squeaking, stains on paint, and bug splats taking paint off, premature uneven brake wear, on 3rd windsh...
1	speed_garmin_nuvi_255W_gps	Another feature on the 255w is a display of the posted speed limit on the road which you are currently on right above your current displayed speed .\n0 I found myself not even looking at my car speedometer as I could easily see my current speed and the speed limit of my route at a glance .\n1 ...
2	rooms_swissotel_chicago	The Swissotel is one of our favorite hotels in Chicago and the corner rooms have the most fantastic views in the city .\n0 The rooms look like they were just remodled and upgraded, there was an HD TV and a nice iHome docking station to put my iPod so I could set the alarm to wake up with my music instead of the radio .\n1

- 파일 이름 자체만으로 의견의 텍스트가 어떤 제품/서비스에 대한 리뷰인지 알 수 있음
- 문서를 TF-IDF 형태로 피쳐 벡터화
- TfidfVectorizer는 Lemmatization 같은 어근 변환 직접 지원x
 - tokenizer 인자에 커스텀 어근 변환 함수 적용 가능

```

from nltk.stem import WordNetLemmatizer
import nltk
import string

remove_punct_dict = dict((ord(punct), None) for punct in string.punctuation)
lemmar = WordNetLemmatizer()

# 입력으로 들어온 token단어들에 대해서 lemmatization 어근 변환.
def LemTokens(tokens):
    return [lemmar.lemmatize(token) for token in tokens]

# TfidfVectorizer 객체 생성 시 tokenizer인자로 해당 함수를 설정하여 lemmatization 적용
# 입력으로 문장을 받아서 stop words 제거-> 소문자 변환 -> 단어 토큰화 -> lemmatization 어근 변환.
def LemNormalize(text):
    return LemTokens(nltk.word_tokenize(text.lower().translate(remove_punct_dict)))

```

```

from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vect = TfidfVectorizer(tokenizer=LemNormalize, stop_words='english' , #
                             ngram_range=(1,2), min_df=0.05, max_df=0.85 )

#opinion_text 컬럼값으로 feature vectorization 수행
feature_vect = tfidf_vect.fit_transform(document_df['opinion_text'])

```

- 문서별 텍스트가 TF-IDF 변환된 피쳐 벡터화 행렬 데이터에 대해 군집화 수행
- 5개 중심 기반으로 어떠한 군집화 되는지 확인

```

from sklearn.cluster import KMeans

# 5개 집합으로 군집화 수행. 예제를 위해 동일한 클러스터링 결과 도출용 random_state=0
km_cluster = KMeans(n_clusters=5, max_iter=10000, random_state=0)
km_cluster.fit(feature_vect)
cluster_label = km_cluster.labels_
cluster_centers = km_cluster.cluster_centers_

```

- 각 데이터별 할당된 군집 레이블을 파일명과 파일 내용 가지고 있는 document_df DataFrame에 'cluster_label'칼럼을 추가해 저장
- 각 파일명은 의견 리뷰에 대한 주제

```
document_df['cluster_label'] = cluster_label
document_df.head()
```

	filename	opinion_text	cluster_label
0	interior_toyota_camry_2007	First of all, the interior has way too many cheap plastic parts like the cheap plastic center piece that houses the clock.\n0 3 blown struts at 30,000 miles, interior trim coming loose and rattling squeaking, stains on paint, and bug splats taking paint off, premature uneven brake wear, on 3rd windsh...	2
1	speed_garmin_nuvi_255W_gps	Another feature on the 255w is a display of the posted speed limit on the road which you are currently on right above your current displayed speed.\n0 I found myself not even looking at my car speedometer as I could easily see my current speed and the speed limit of my route at a glance.\n1 ...	0
2	rooms_swissotel_chicago	The Swissotel is one of our favorite hotels in Chicago and the corner rooms have the most fantastic views in the city.\n0 The rooms look like they were just remodled and upgraded, there was an HD TV and a nice iHome docking station to put my iPod so I could set the alarm to wake up with my music instead of the radio.\n1 ...	3
3	speed_garmin_nuvi_255W_gps	Another feature on the 255w is a display of the posted speed limit on the road which you are currently on right above your current displayed speed.\n0 I found myself not even looking at my car speedometer as I could easily see my current speed and the speed limit of my route at a glance.\n1 ...	0
4	comfort_toyota_camry_2007	Ride seems comfortable and gas mileage fairly good averaging 26 city and 30 open road.\n0 Seats are fine, in fact of all the smaller sedans this is the most comfortable I found for the price as I am 6', 2 and 250#.\n1 Great gas mileage and comfortable on long trips ...	2

- 판다스 DataFrame의 sort_claues를 수행
 - 인자로 입력된 정렬칼럼명으로 데이터 정렬 가능
- document_df DataFrame 객체에서 cluster_label로 어떤 파일명으로 매칭됐는지 보면서 군집화 결과 확인


```
document_df[document_df['cluster_label']==0].sort_values(by='filename')
```

	filename	opinion_text	cluster_label
65	accuracy_garmin_nuvi_255W_gps	, and is very, very accurate .\n0 but for the most part, we find that the Garmin software provides accurate directions, wherever we intend to go .\n1 This functi...	0
91	accuracy_garmin_nuvi_255W_gps	, and is very, very accurate .\n0 but for the most part, we find that the Garmin software provides accurate directions, wherever we intend to go .\n1 This functi...	0
25	directions_garmin_nuvi_255W_gps	You also get upscale features like spoken directions including street names and programmable POIs .\n0 I used to hesitate to go out of my directions but no...	0
38	directions_garmin_nuvi_255W_gps	You also get upscale features like spoken directions including street names and programmable POIs .\n0 I used to hesitate to go out of my directions but no...	0
64	display_garmin_nuvi_255W_gps	3 quot widescreen display was a bonus .\n0 This made for smoother graphics on the 255w of the vehicle moving along displayed roads, where the 750's display was more of a jerky movement .\n1 ...	0
90	display_garmin_nuvi_255W_gps	3 quot widescreen display was a bonus .\n0 This made for smoother graphics on the 255w of the vehicle moving along displayed roads, where the 750's display was more of a jerky movement .\n1 ...	0
18	satellite_garmin_nuvi_255W_gps	It's fast to acquire satellites .\n0 If you've ever had a Brand X GPS take you on some strange route that adds 20 minutes to your trip, has you turn the wrong way down a one way road, tell you to turn AFTER you've passed the street, frequently loses the satellite signal, or has old maps missing streets, you know how important this stuff is .\n1 ...	0
84	satellite_garmin_nuvi_255W_gps	It's fast to acquire satellites .\n0 If you've ever had a Brand X GPS take you on some strange route that adds 20 minutes to your trip, has you turn the wrong way down a one way road, tell you to turn AFTER you've passed the street, frequently loses the satellite signal, or has old maps missing streets, you know how important this stuff is .\n1 ...	0
28	screen_garmin_nuvi_255W_gps	It is easy to read and when touching the screen it works great !\n0 and zoom out buttons on the 255w to the same side of the screen which makes it a bit easier .\n1 ...	0

- cluster #0은 호텔에 대한 리뷰로 군집화

```
document_df[document_df['cluster_label']==1].sort_values(by='filename')
```

76	price_amazon_kindle	If a case was included, as with the Kindle 1, that would have been reflected in a higher price .\n0 lower overall price, with nice leather cover .\n1 ...	1
22	price_amazon_kindle	If a case was included, as with the Kindle 1, that would have been reflected in a higher price .\n0 lower overall price, with nice leather cover .\n1 ...	1
92	screen_ipod_nano_8gb	As always, the video screen is sharp and bright .\n0 2, inch screen and a glossy, polished aluminum finish that one CNET editor described as looking like a Christmas tree ornament .\n1 ...	1
7	screen_ipod_nano_8gb	As always, the video screen is sharp and bright .\n0 2, inch screen and a glossy, polished aluminum finish that one CNET editor described as looking like a Christmas tree ornament .\n1 ...	1
30	screen_netbook_1005ha	Keep in mind that once you get in a room full of light or step outdoors screen reflections could become annoying .\n0 I've used mine outsi...	1
75	screen_netbook_1005ha	Keep in mind that once you get in a room full of light or step outdoors screen reflections could become annoying .\n0 I've used mine outsi...	1
44	size_asus_netbook_1005ha	A few other things I'd like to point out is that you must push the micro, sized right angle end of the ac adapter until it snaps in place or the battery may not charge .\n0 The full size right shift k...	1
94	size_asus_netbook_1005ha	A few other things I'd like to point out is that you must push the micro, sized right angle end of the ac adapter until it snaps in place or the battery may not charge .\n0 The full size right shift k...	1
42	sound_ipod_nano_8gb	headphone jack i got a clear case for it and it i got a clear case for it and it like prvents me from being able to put the jack all the way in so the sound can b messssed up or i can get it in there and its playing well them go to move or something and it slides out .\n0 Picture and sound quality are excellent for this typ of devic .\n1 ...	1
59	sound_ipod_nano_8gb	headphone jack i got a clear case for it and it i got a clear case for it and it like prvents me from being able to put the jack all the way in so the sound can b messssed up or i can get it in there and its playing well them go to move or something and it slides out .\n0 Picture and sound quality are excellent for this typ of devic .\n1 ...	1
I bought the 8, gig Ipod Nano that has the built, in video			

- cluster #1은 포터블 전자기기 및 주요 구성요소에 대한 리뷰로 군집화

```
document_df[document_df['cluster_label']==2].sort_values(by='filename')
```

43	performance_honda_accord_2008	cruiser .\n0 6, 4, 3 eco engine has poor performance and gas mileage of 22 highway .\n1 Overall performance is good but comfort level is poor .\n2 ...	2
34	performance_honda_accord_2008	Very happy with my 08 Accord, performance is quite adequate it has nice looks and is a great long, distance cruiser .\n0 6, 4, 3 eco engine has poor performance and gas mileage of 22 highway .\n1 Overall performance is good but comfort level is poor .\n2 ...	2
73	performance_netbook_1005ha	The Eee Super Hybrid Engine utility lets users overclock or underclock their Eee PC's to boost performance or provide better battery life depending on their immediate requirements .\n0 In Super Performance mode CPU, Z shows the bus speed to increase up to 169 .\n1 One...	2
101	performance_netbook_1005ha	The Eee Super Hybrid Engine utility lets users overclock or underclock their Eee PC's to boost performance or provide better battery life depending on their immediate requirements .\n0 In Super Performance mode CPU, Z shows the bus speed to increase up to 169 .\n1 One...	2
66	quality_toyota_camry_2007	I previously owned a Toyota 4Runner which had incredible build quality and reliability .\n0 I bought the Camry because of Toyota reliability and qua...	2
33	quality_toyota_camry_2007	I previously owned a Toyota 4Runner which had incredible build quality and reliability .\n0 I bought the Camry because of Toyota reliability and qua...	2
63	seats_honda_accord_2008	Front seats are very uncomfortable .\n0 No memory seats, no trip computer, can only display outside temp with trip odometer .\n1 ...	2
68	seats_honda_accord_2008	Front seats are very uncomfortable .\n0 No memory seats, no trip computer, can only display outside temp with trip odometer .\n1 ...	2
40	speed_windows7	Windows 7 is quite simply faster, more stable, boots faster, goes to sleep faster, comes back from sleep faster, manages your files better and on top of that it's beautiful to look at and easy to use .\n0 , faster about 20% to 30% faster at running applications than my Vista , seriously\n1 ...	2
50	speed_windows7	Windows 7 is quite simply faster, more stable, boots faster, goes to sleep faster, comes back from sleep faster, manages your files better and on top of that it's beautiful to look at and easy to use .\n0 . faster about	2

- cluster #2는 cluster #1과 비슷,
 - 주로 차량용 내비게이션으로 군집 구성

```
document_df[document_df['cluster_label']==3].sort_values(by='filename')
```

97	rooms_bestwestern_hotel_sfo	Great Location , Nice Rooms , H...	3
2	rooms_swissotel_chicago	The Swissotel is one of our favorite hotels in Chicago and the corner rooms have the most fantastic views in the city .\n0 The rooms look like they were just remodeled and upgraded, there was an HD TV and a nice iHome docking station to put my iPod so I could set the alarm to wake up with my music instead of the radio .\n1 ...	3
12	rooms_swissotel_chicago	The Swissotel is one of our favorite hotels in Chicago and the corner rooms have the most fantastic views in the city .\n0 The rooms look like they were just remodeled and upgraded, there was an HD TV and a nice iHome docking station to put my iPod so I could set the alarm to wake up with my music instead of the radio .\n1 ...	3
58	service_bestwestern_hotel_sfo	Both of us having worked in tourism for over 14 years were very disappointed at the level of service provided by this gentleman .\n0 The service was good, very friendly staff and we loved the free wine reception each night .\n1 ...	3
98	service_bestwestern_hotel_sfo	Both of us having worked in tourism for over 14 years were very disappointed at the level of service provided by this gentleman .\n0 The service was good, very friendly staff and we loved the free wine reception each night .\n1 ...	3
78	service_holiday_inn_london	not customer, oriented hotelvery low service levelboor reception\n0 The room was quiet, clean, the bed and pillows were comfortable, and the serv...	3
11	service_holiday_inn_london	not customer, oriented hotelvery low service levelboor reception\n0 The room was quiet, clean, the bed and pillows were comfortable, and the serv...	3
52	service_swissotel_hotel_chicago	Mediocre room and service for a very extravagant price .\n0 ...	3
17	service_swissotel_hotel_chicago	Mediocre room and service for a very extravagant price .\n0 ...	3
23	staff_bestwestern_hotel_sfo	Staff are friendl...	3
95	staff_bestwestern_hotel_sfo	Staff are friendl...	3
		The staff at Swissotel were not particularly nice .\n0 Each time I waited at the counter for staff for several	

- cluster #3는 킨들 리뷰가 하나 섞여있긴 함
- cluster #0과 같이 대부분 호텔에 대한 리뷰로 군집화

```
document_df[document_df['cluster_label']==4].sort_values(by='filename')
```

	filename	opinion_text	cluster_label
29	location_bestwestern_hotel_sfo	Good Value good location , ideal choice .\n0 Great Location , Nice Rooms , Helpless Concierge\n1 ...	4
81	location_bestwestern_hotel_sfo	Good Value good location , ideal choice .\n0 Great Location , Nice Rooms , Helpless Concierge\n1 ...	4
56	location_holiday_inn_london	Great location for tube and we crammed in a fair amount of sightseeing in a short time .\n0 All in all, a normal chain hotel on a nice lo...	4
57	location_holiday_inn_london	Great location for tube and we crammed in a fair amount of sightseeing in a short time .\n0 All in all, a normal chain hotel on a nice lo...	4
35	price_holiday_inn_london	All in all, a normal chain hotel on a nice location , I will be back if I do not find anything closer to Picadilly for a better price .\n0 ...	4
41	price_holiday_inn_london	All in all, a normal chain hotel on a nice location , I will be back if I do not find anything closer to Picadilly for a better price .\n0 ...	4

- cluster #4는 자동차에 대한 리뷰로 군집화

```

from sklearn.cluster import KMeans

# 3개의 집합으로 군집화
km_cluster = KMeans(n_clusters=3, max_iter=10000, random_state=0)
km_cluster.fit(feature_vect)
cluster_label = km_cluster.labels_

# 소속 클러스터를 cluster_label 컬럼으로 할당하고 cluster_label 값으로 정렬
document_df['cluster_label'] = cluster_label
document_df.sort_values(by='cluster_label')

```

	filename	opinion_text	cluster_label
1	speed_garmin_nuvi_255W_gps	Another feature on the 255w is a display of the posted speed limit on the road which you are currently on right above your current displayed speed .\n0 I found myself not even looking at my car speedometer as I could easily see my current speed and the speed limit of my route at a glance .\n1 ...	0
3	speed_garmin_nuvi_255W_gps	Another feature on the 255w is a display of the posted speed limit on the road which you are currently on right above your current displayed speed .\n0 I found myself not even looking at my car speedometer as I could easily see my current speed and the speed limit of my route at a glance .\n1 ...	0
10	free_bestwestern_hotel_sfo	The wine reception is a great idea as it is nice to meet other travellers and great having access to the free Internet access in our room .\n0 They also have a computer available with free internet which is a nice bonus but I didn't find that out till the day before we left but was still able to get on there to check our flight to Vegas the next day .\n1 ...	0
18	satellite_garmin_nuvi_255W_gps	It's fast to acquire satellites .\n0 If you've ever had a Brand X GPS take you on some strange route that adds 20 minutes to your trip, has you turn the wrong way down a one way road, tell you to turn AFTER you've passed the street, frequently loses the satellite signal, or has old maps missing streets, you know how important this stuff is .\n1 ...	0
25	directions_garmin_nuvi_255W_gps	You also get upscale features like spoken directions including street names and programmable POIs .\n0 I used to hesitate to go out of my directions but no...	0
...	...	...	...

- cluster #0은 포터블 전자기기 리뷰로만 군집화
- cluster #1은 자동차 리뷰로만 군집화
- cluster #2는 호텔 리뷰로만 군집화
- 각 군집을 구성하는 핵심 단어가 어떤 것이 있는지 확인
- KMeans 객체는 각 군집 구성 단어 피처가 군집 중심 기준 얼마나 가깝게 위치해 있는지 clusters_centers_ 속성 제공
 - 배열 값으로 제공
 - 행은 개별 군집
 - 열은 개별 피처

- 각 배열 내의 값은 개별 군집 내의 상대 위치를 숫자 값으로 표현한 일종의 좌표 값

```
cluster_centers = km_cluster.cluster_centers_
print('cluster_centers shape :', cluster_centers.shape)
print(cluster_centers)

cluster_centers shape : (3, 4610)
[[0.00858908 0.          0.          ... 0.01533606 0.          0.          ]
 [0.00905105 0.          0.          ... 0.00250991 0.          0.          ]
 [0.00112301 0.00099043 0.00109661 ... 0.          0.00115204 0.00091017]]
```

- cluster_centers_는 (3, 4611)배열
 - 군집 3개, word 피쳐 4611개로 구성
- 각 행의 배열
 - 각 군집 내의 4611개 피쳐의 위치가 개별 중심과 얼마나 가까운가 상대 값으로 나타낸 것
- cluster_centers_ 속성값을 이용해 각 군집별 핵심 단어 찾기
- ndarray의 argsort()[::-1] 이용하면 cluster_centers 배열 내 값 큰 순으로 정렬된 위치 인덱스값 반환
- get_cluster_details() 생성해 위에 대한 처리 담당

```

# 군집별 top n 핵심단어, 그 단어의 중심 위치 상대값, 대상 파일명들을 반환함.
def get_cluster_details(cluster_model, cluster_data, feature_names, clusters_num, top_n_features=10):
    cluster_details = {}

    # cluster_centers array 의 값이 큰 순으로 정렬된 index 값을 반환
    # 군집 중심점(centroid)별 할당된 word 피쳐들의 거리값이 큰 순으로 값을 구하기 위함.
    centroid_feature_ordered_ind = cluster_model.cluster_centers_.argsort()[:, :-1]

    # 개별 군집별로 iteration하면서 핵심단어, 그 단어의 중심 위치 상대값, 대상 파일명 입력
    for cluster_num in range(clusters_num):
        # 개별 군집별 정보를 담은 데이터 초기화.
        cluster_details[cluster_num] = {}
        cluster_details[cluster_num]['cluster'] = cluster_num

        # cluster_centers_.argsort()[:, :-1] 로 구한 index 를 이용하여 top n 피쳐 단어를 구함.
        top_feature_indexes = centroid_feature_ordered_ind[cluster_num, :top_n_features]
        top_features = [ feature_names[ind] for ind in top_feature_indexes ]

        # top_feature_indexes를 이용해 해당 피쳐 단어의 중심 위치 상대값 구함
        top_feature_values = cluster_model.cluster_centers_[cluster_num, top_feature_indexes].tolist()

        # cluster_details 딕셔너리 객체에 개별 군집별 핵심 단어와 중심위치 상대값, 그리고 해당 파일명 입력
        cluster_details[cluster_num]['top_features'] = top_features
        cluster_details[cluster_num]['top_features_value'] = top_feature_values
        filenames = cluster_data[cluster_data['cluster_label'] == cluster_num]['filename']
        filenames = filenames.values.tolist()
        cluster_details[cluster_num]['filenames'] = filenames

    return cluster_details

```

- get_cluster_details()를 호출하면 dictionary를 원소로 가지는 리스트인 cluster_details를 반환
 - 개별 군집번호, 핵심 단어, 핵심 단어 중심 위치 상대값, 파일명 속성 값 정보 존재
- 별도의 print_cluster_details() 함수 생성
- get_cluster_details(), print_cluster_details()를 호출
- 피쳐명 리스트는 TF-IDF 변환된 tfidf_vect 객체에서 get_feature_names()로 추출


```
def print_cluster_details(cluster_details):
    for cluster_num, cluster_detail in cluster_details.items():
        print('##### Cluster: {}'.format(cluster_num))
        print('Top features:', cluster_detail['top_features'])
        print('Reviews 파일명 :', cluster_detail['filenames'][:7])
        print('=====')

feature_names = tfidf_vect.get_feature_names_out()

cluster_details = get_cluster_details(cluster_model=km_cluster, cluster_data=document_df,
                                     feature_names=feature_names, clusters_num=3, top_n_features=10)
print_cluster_details(cluster_details)

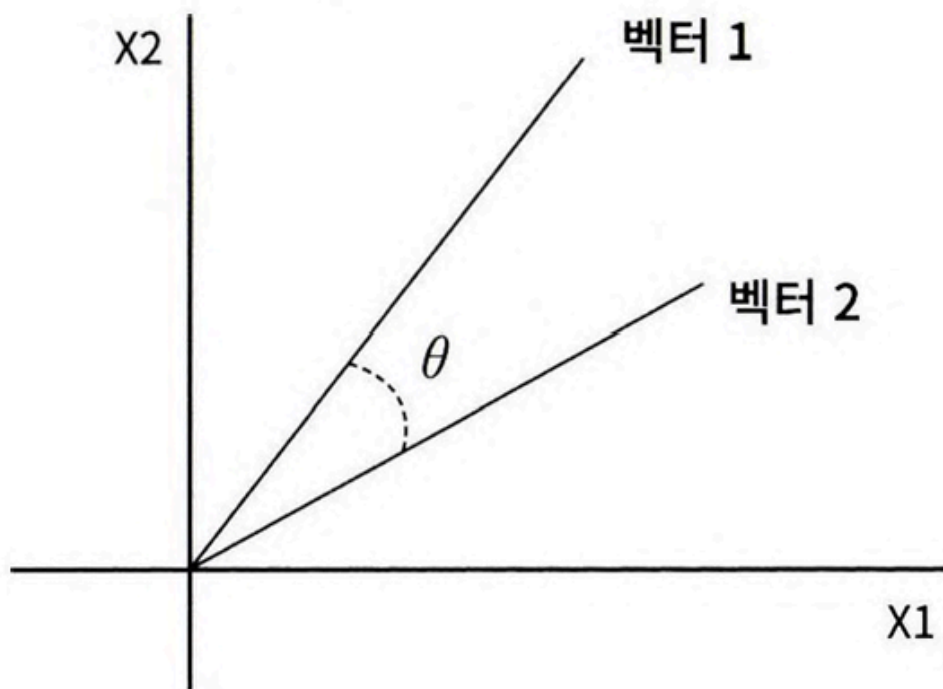
##### Cluster 0
Top features: ['direction', 'map', 'voice', 'screen', 'speed limit', 'accurate', 'satellite', 'speed', 'limit', 'u
Reviews 파일명 : ['speed_garmin_nuvi_255W_gps', 'speed_garmin_nuvi_255W_gps', 'free_bestwestern_hotel_sfo', 'satel
=====
##### Cluster 1
Top features: ['battery', 'screen', 'battery life', 'keyboard', 'life', 'kindle', 'video', 'size', 'page', 'button
Reviews 파일명 : ['screen_ipod_nano_8gb', 'features_windows7', 'buttons_amazon_kindle', 'battery-life_ipod_nano_8g
=====
##### Cluster 2
Top features: ['room', 'hotel', 'service', 'staff', 'interior', 'food', 'location', 'seat', 'mileage', 'comfortabl
Reviews 파일명 : ['interior_toyota_camry_2007', 'rooms_swissotel_chicago', 'comfort_toyota_camry_2007', 'parking_b
=====
```

- Cluster #0은 화면과 배터리 수명 등이 핵심 단어로 군집화
- Cluster #1은 실내 인테리어, 좌석, 연료 효율 등이 핵심 단어로 군집화
- Cluster #2는 방과 서비스 등이 핵심 단어로 군집화

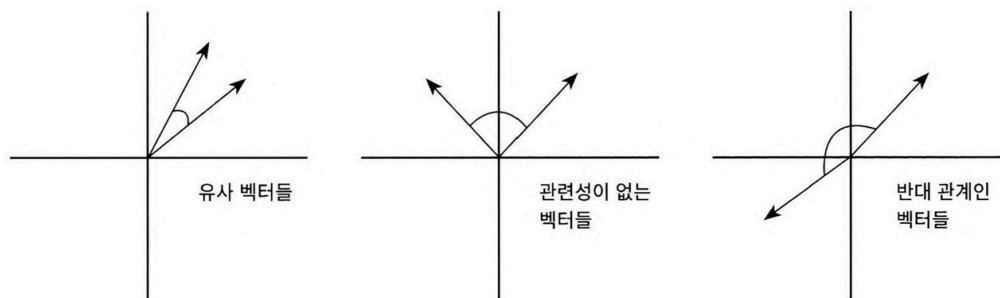
08 문서 유사도

문서 유사도 측정 방법 - 코사인 유사도

- 벡터와 벡터 간의 유사도 계산 시
 - 벡터 크기보다 벡터의 상호 방향성이 얼마나 유사한지에 기반
 - ⇒ 두 벡터 사이의 사잇각을 구해서 얼마나 유사한지 수치로 적용한 것



- 두 벡터의 사잇각에 따라 상호 관계는 유사/무관/반대 될 수 있음



- 두 벡터 내적 값은 두 벡터의 크기를 곱한 값의 코사인 각도 값을 곱한 것
- $\Rightarrow$ 유사도 코사인은 두 벡터의 내적을 총 벡터 크기의 합으로 나눈 것
- = 내적 결과를 총 벡터 크기로 정규화(L2 Norm)한 것

$$A * B = \|A\| \|B\| \cos \theta$$

$$\text{similarity} = \cos \theta = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

- 문서를 피처 벡터화변환하면 차원이 매우 많은 희소 행렬되기 쉬움
- 희소 행렬 기반에서 문서와 문서 벡터 간의 크기에 기반한 유사도 지표는정확도 떨어지기 쉬움
- ⇒ 코사인 유사도 가장 많이 사용됨

- 두 개의 넘파이 배열에 대한 코사인 유사도 구하는 cos_similarity() 함수 생성
- doc_list로 정의된 3개의 간단한 문서의 유사도 비교 위해 TF-IDF로 벡터화된 행렬로 변환

```
import numpy as np

def cos_similarity(v1, v2):
    dot_product = np.dot(v1, v2)
    l2_norm = (np.sqrt(sum(np.square(v1))) * np.sqrt(sum(np.square(v2))))
    similarity = dot_product / l2_norm

    return similarity

from sklearn.feature_extraction.text import TfidfVectorizer

doc_list = ['if you take the blue pill, the story ends' ,
            'if you take the red pill, you stay in Wonderland',
            'if you take the red pill, I show you how deep the rabbit hole goes']

tfidf_vect_simple = TfidfVectorizer()
feature_vect_simple = tfidf_vect_simple.fit_transform(doc_list)
print(feature_vect_simple.shape)

(3, 18)
```

- 반환된 행렬은 희소 행렬
 - cos_similarity() 함수의 인자인 array로 만들기 위해 밀집 행렬로 변환한 뒤 다시 각각을 배열로 변환
- cos_similarity() 함수 이용하여 두 개 문서의 유사도 측정

```
# TfidfVectorizer로 transform()한 결과는 Sparse Matrix이므로 Dense Matrix로 변환.
feature_vect_dense = feature_vect_simple.todense()

#첫번째 문장과 두번째 문장의 feature vector 추출
vect1 = np.array(feature_vect_dense[0]).reshape(-1,)
vect2 = np.array(feature_vect_dense[1]).reshape(-1,)

#첫번째 문장과 두번째 문장의 feature vector로 두개 문장의 Cosine 유사도 추출
similarity_simple = cos_similarity(vect1, vect2 )
print('문장 1, 문장 2 Cosine 유사도: {0:.3f}'.format(similarity_simple))
```

문장 1, 문장 2 Cosine 유사도: 0.402

- 첫 번째 문장과 세 번째 문장, 두 번째 문장과 세 번째 문장의 유사도 측정

```
vect1 = np.array(feature_vect_dense[0]).reshape(-1,)
vect3 = np.array(feature_vect_dense[2]).reshape(-1,)
similarity_simple = cos_similarity(vect1, vect3 )
print('문장 1, 문장 3 Cosine 유사도: {0:.3f}'.format(similarity_simple))

vect2 = np.array(feature_vect_dense[1]).reshape(-1,)
vect3 = np.array(feature_vect_dense[2]).reshape(-1,)
similarity_simple = cos_similarity(vect2, vect3 )
print('문장 2, 문장 3 Cosine 유사도: {0:.3f}'.format(similarity_simple))
```

문장 1, 문장 3 Cosine 유사도: 0.404
문장 2, 문장 3 Cosine 유사도: 0.456

- 사이킷런은 코사인 유사도 측정 위해 API 제공
 - `sklearn.metrics.pairwise.cosine_similarity`
- 첫 번째 문서와 비교해 바로 자신 문서인 첫 번째 문서,
- 두 번째, 세 번째 문서의 유사도 측정

```

from sklearn.metrics.pairwise import cosine_similarity

similarity_simple_pair = cosine_similarity(feature_vect_simple[0] , feature_vect_simple)
print(similarity_simple_pair)

[[1.          0.40207758  0.40425045]]

from sklearn.metrics.pairwise import cosine_similarity

similarity_simple_pair = cosine_similarity(feature_vect_simple[0] , feature_vect_simple[1:])
print(similarity_simple_pair)

[[0.40207758  0.40425045]]

similarity_simple_pair = cosine_similarity(feature_vect_simple , feature_vect_simple)
print(similarity_simple_pair)
print('shape:',similarity_simple_pair.shape)

[[1.          0.40207758  0.40425045]
 [0.40207758  1.          0.45647296]
 [0.40425045  0.45647296  1.          ]]
shape: (3, 3)

```

- cosine_similarity()는 쌍으로 (pair) 코사인 유사도 값을 제공할 수 있음
- 모든 개별 문서에 쌍으로 코사인 유사도 값 계산

Opinion Review 데이터 세트를 이용한 문서 유사도 측정

- 호텔을 주제로 군집화된 문서 이용해 특정 문서와 다른 문서 간의 유사도 알아봄
- 문서를 피처벡터화해 변환하면 문서 내 단어 출현 빈도 같은 값을 부여해 각 문서가 단어 피처의 값으로 벡터화됨

```

import pandas as pd
import glob, os
import warnings
warnings.filterwarnings('ignore')
pd.set_option('display.max_colwidth', 700)

# 업로드한 파일들은 모두 /content에 저장됨
path = '/content'
all_files = glob.glob(os.path.join(path, "*.data"))
filename_list = []
opinion_text = []

for file_ in all_files:
    # 파일을 table 형식으로 읽을 수 있을 때
    try:
        df = pd.read_table(file_, index_col=None, header=0, encoding='latin1')
        content = df.to_string()
    except:
        # 파일이 표 형식이 아니면 그냥 텍스트로 읽기
        with open(file_, encoding='latin1') as f:
            content = f.read()
        filename_ = file_.split('/')[1]
        filename = filename_.split('.')[0]
        filename_list.append(filename)
        opinion_text.append(content)

document_df = pd.DataFrame({'filename': filename_list, 'opinion_text': opinion_text})

tfidf_vect = TfidfVectorizer(tokenizer=LemNormalize, stop_words='english', #
                             ngram_range=(1,2), min_df=0.05, max_df=0.85 )
feature_vect = tfidf_vect.fit_transform(document_df['opinion_text'])

km_cluster = KMeans(n_clusters=3, max_iter=10000, random_state=0)
km_cluster.fit(feature_vect)
cluster_label = km_cluster.labels_
cluster_centers = km_cluster.cluster_centers_
document_df['cluster_label'] = cluster_label

```

- 각 문서가 피처 벡터화된 데이터를 cosine_similarity()를 이용해 상호 비교, 유사도 확인

```

from nltk.stem import WordNetLemmatizer
import nltk
import string

remove_punct_dict = dict((ord(punct), None) for punct in string.punctuation)
lemmar = WordNetLemmatizer()

# 입력으로 들어온 token단어들에 대해서 lemmatization 어근 변환.
def LemTokens(tokens):
    return [lemmar.lemmatize(token) for token in tokens]

# TfidfVectorizer 객체 생성 시 tokenizer인자로 해당 함수를 설정하여 lemmatization 적용
# 입력으로 문장을 받아서 stop words 제거-> 소문자 변환 -> 단어 토큰화 -> lemmatization 어근 변환.
def LemNormalize(text):
    return LemTokens(nltk.word_tokenize(text.lower().translate(remove_punct_dict)))

```

- 데이터 추출, 해당하는 TfidfVectorizer의 데이터 추출
- 호텔 군집화 데이터를 기반으로 별도의 TF-IDF 벡터화를 수행하지 않고 만들어진 데이터에서 그대로 추출
- DataFrame 객체 변수인 document_df에서 먼저 호텔로 군집화된 문서의 인덱스 추출
- 추출된 인덱스 그대로 이용해 TfidfVectorizer 객체 변수인 feature_vect에서 호텔로 군집화된 문서의 피쳐 벡터를 추출

```

from sklearn.metrics.pairwise import cosine_similarity

# cluster_label=2인 데이터는 호텔로 클러스터링된 데이터임. DataFrame에서 해당 Index를 추출
hotel_indexes = document_df[document_df['cluster_label']==2].index
print('호텔로 클러스터링 된 문서들의 DataFrame Index:', hotel_indexes)

# 호텔로 클러스터링된 데이터 중 첫번째 문서를 추출하여 파일명 표시.
comparison_docname = document_df.iloc[hotel_indexes[0]]['filename']
print('##### 비교 기준 문서명 ', comparison_docname, ' 와 타 문서 유사도#####')

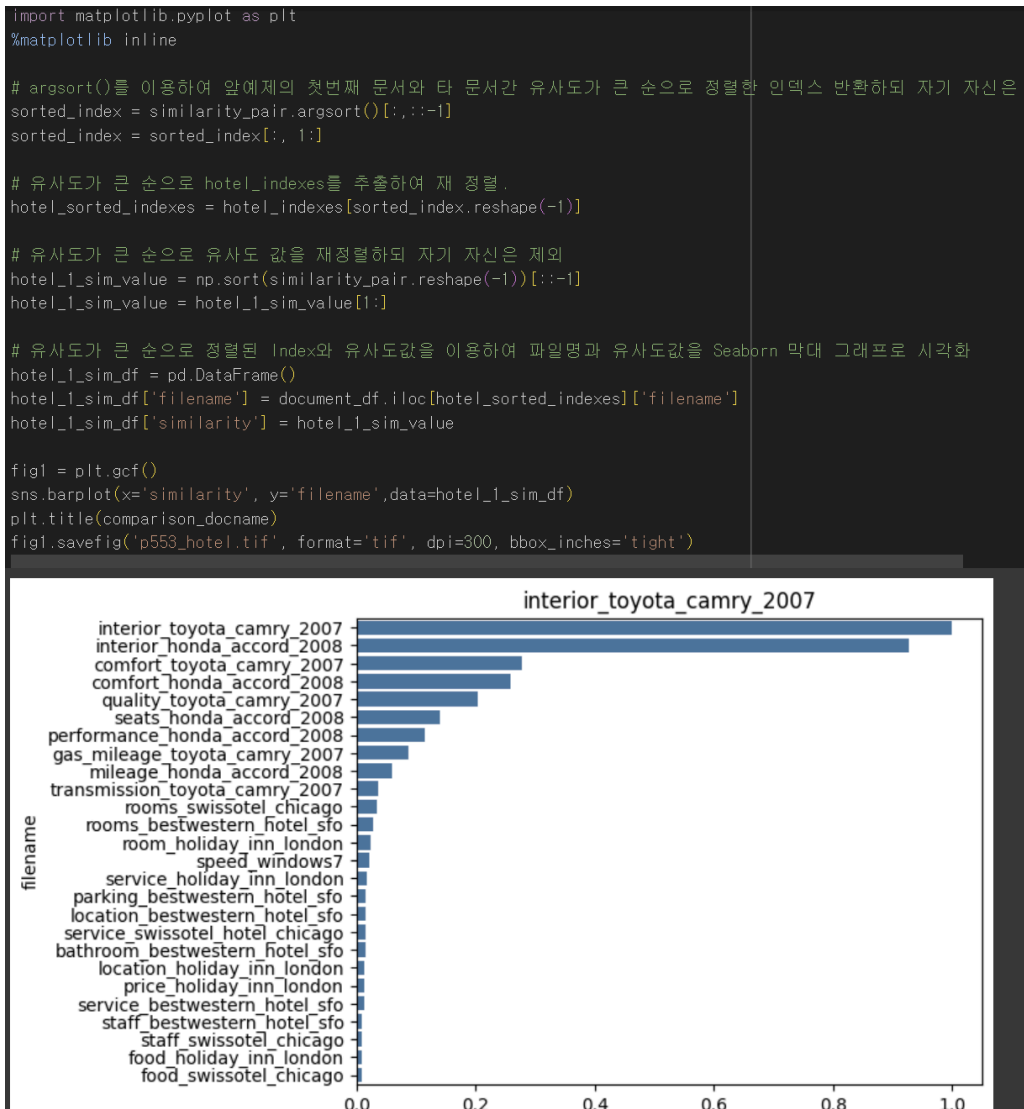
''' document_df에서 추출한 Index 객체를 feature_vect로 입력하여 호텔 클러스터링된 feature_vect 추출
이를 이용하여 호텔로 클러스터링된 문서 중 첫번째 문서와 다른 문서간의 유사도 측정.'''
similarity_pair = cosine_similarity(feature_vect[hotel_indexes[0]], feature_vect[hotel_indexes])
print(similarity_pair)

호텔로 클러스터링 된 문서들의 DataFrame Index: Index([ 0,  2,  4,  5,  6, 11, 12, 13, 14, 15, 17, 19, 23, 26, 27,
33, 34, 35, 36, 39, 40, 41, 43, 45, 47, 48, 49, 50, 52, 53, 56, 57, 58,
61, 63, 66, 67, 68, 69, 71, 78, 79, 81, 82, 85, 86, 95, 97, 98],
dtype='int64')
##### 비교 기준 문서명 interior_toyota_camry_2007 와 타 문서 유사도#####
[[1. 0.03514439 0.27806853 0.01604357 0.00896711 0.01693995
 0.03514439 0.02785844 0.06061944 0.25919097 0.01505358 0.01457674
 0.0094297 0.00828266 0.02413097 0.01528842 0.03667727 0.00884942
 0.20454454 0.11424159 0.01356376 0.00896711 0.00884942 0.02169188
 0.01356376 0.11424159 0.06061944 1. 0.02413097 0.92795466
 0.02169188 0.01505358 0.01604357 0.01405917 0.01405917 0.0125491
 0.08765973 0.13920666 0.20454454 0.92795466 0.13920666 0.08765973
 0.25919097 0.01693995 0.01457674 0.01528842 0.03667727 0.00828266
 0.27806853 0.0094297 0.02785844 0.0125491 ]]

```

- 단순 숫자만으로는 문서가 어느 정도 유사한지 직관적으로 이해하기 힘들

- 첫 번째 문서와 다른 문서 간에 유사도 높은 순으로 정렬, 시각화
- cosine_similarity()는 쌍 형태의 ndarray를 반환
 - 판다스 인덱스로 이용하기 위해 reshape(-1)로 차원 변경



- 첫 번째 문서인 샌프란시스코의 베스트 웨스턴 호텔 화장실 리뷰인 bathroom_bestweterern_hotel_sfo와 가장 비슷한 문서는 room_holiday_inn_london

09 한글 텍스트 처리 - 네이버 영화 평점 감성 분석

- 한글은 영어 등 라틴어 계열보다 처리 어려움
 - 띄어쓰기와 다양한 조사 때문
 - 의미가 왜곡될 가능성이 있음
 - 영어보다 띄어쓰기가 훨씬 어려움
 - 조사는 어근 추출 등의 전처리 시 제거하기 까다로움
- KoNLPy는 파이썬 대표적인 한글 형태소 패키지
- 형태소 분석(Morphological analysis)
 - 말뭉치를 형태소 어근 단위로 쪼개고 각 형태소에 POS tagging 부착하는 작업
- 학습 데이터 세트의 0과 1의 Lable 값 비율
- 1이 긍정 0이 부정

```
import pandas as pd
```

```
train_df = pd.read_csv('ratings_train.txt', sep='##t')  
train_df.head(3)
```

	id	document	label
0	9976970	아 더빙.. 진짜 짜증나네요 목소리	0
1	3819312	흠...포스터보고 초딩영화줄....오버연기조차 가볍지 않구나	1
2	10265843	너무재밌었다그래서보는것을추천한다	0

```
train_df['label'].value_counts( )
```

	count
label	
0	75173
1	74827

```
dtype: int64
```

- train_df 경우 리뷰 텍스트 가지는 document 칼럼에 Null이 일부 존재하므로 이 값을 공백으로 변환
- 문자가 아닌 숫자의 경우 단어적 의미로 부족
- 파이썬의 정규 표현식 모듈인 re 이용해 공백으로 변환
- 테스트셋도 동일한 데이터 가공 수행

```
import re

train_df = train_df.fillna(' ')
# 정규 표현식을 이용하여 숫자를 공백으로 변경(정규 표현식으로 \d 는 숫자를 의미함.)
train_df['document'] = train_df['document'].apply( lambda x : re.sub(r"#d+", " ", x) )

# 테스트 데이터 셋을 로딩하고 동일하게 Null 및 숫자를 공백으로 변환
test_df = pd.read_csv('ratings_test.txt', sep='#t')
test_df = test_df.fillna(' ')
test_df['document'] = test_df['document'].apply( lambda x : re.sub(r"#d+", " ", x) )

# id 칼럼 삭제 수행
train_df.drop('id', axis=1, inplace=True)
test_df.drop('id', axis=1, inplace=True)
```

- TF-IDF 방식으로 단어 벡터화
- 각 문장을 한글 형태소 분석 통해 형태소 단어로 토큰화
- 한글 형태소 엔진은 Okt 클래스 이용
- Okt 객체의 morphs() 메서드 이용하면 문장을 형태소 단어 형태로 토큰화해 list 객체로 반환

```
!pip install konlpy

Collecting konlpy
  Downloading konlpy-0.6.0-py2.py3-none-any.whl.metadata (1.9 kB)
Collecting JPype1>=0.7.0 (from konlpy)
  Downloading jpype1-1.5.2-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (4.9 kB)
Requirement already satisfied: lxml>=4.1.0 in /usr/local/lib/python3.11/dist-packages (from konlpy) (5.4.0)
Requirement already satisfied: numpy>=1.6 in /usr/local/lib/python3.11/dist-packages (from konlpy) (2.0.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from JPype1>=0.7.0->konlpy)
Downloading konlpy-0.6.0-py2.py3-none-any.whl (19.4 MB)
----- 19.4/19.4 MB 62.7 MB/s eta 0:00
Downloading jpype1-1.5.2-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (494 kB)
----- 494.1/494.1 kB 27.1 MB/s eta 0:00

Installing collected packages: JPype1, konlpy
Successfully installed JPype1-1.5.2 konlpy-0.6.0

from konlpy.tag import Okt

okt = Okt()
def tw_tokenizer(text):
    tokens_ko = okt.morphs(text)
    return tokens_ko
```

- 사이킷런의 TfidfVectorizer 이용해 TF-IDF 피쳐 모델 생성
- ngram(1,2), min_df=3, max_df는 상위 90%로 제한

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

# Twitter 객체의 morphs() 객체를 이용한 tokenizer를 사용. ngram_range는 (1,2)
tfidf_vect = TfidfVectorizer(tokenizer=tw_tokenizer, ngram_range=(1,2), min_df=3, max_df=0.9)
tfidf_vect.fit(train_df['document'])
tfidf_matrix_train = tfidf_vect.transform(train_df['document'])
```

- 로지스틱 회귀 이용해 분류 기반의 감성 분석 수행
- 로지스틱 회귀의 하이퍼 파라미터 C의 최적화 위해 GridSearchCV 이용

```
# Logistic Regression 을 이용하여 감성 분석 Classification 수행.
lg_clf = LogisticRegression(random_state=0, solver='liblinear')

# Parameter C 최적화를 위해 GridSearchCV 를 이용.
params = { 'C': [1, 3.5, 4.5, 5.5, 10] }
grid_cv = GridSearchCV(lg_clf, param_grid=params, cv=3, scoring='accuracy', verbose=1)
grid_cv.fit(tfidf_matrix_train, train_df['label'])
print(grid_cv.best_params_, round(grid_cv.best_score_, 4))

Fitting 3 folds for each of 5 candidates, totalling 15 fits
{'C': 3.5} 0.8593
```

- C가 3.5일 때 최고 0.8593 정확도
- 테스트 셋으로 예측할 때는 학습할 때 적용한 TfidfVectorizer를 그대로 사용
 - 학습 시 설정된 TfidfVectorizer의 피처 개수와 테스트 데이터를 TfidfVectorizer로 변환할 피처 개수 같게 하기 위해
- 학습 데이터에 사용된 TfidfVectorizer 객체 변수인 tfidf_vect 이용해 transform()을 테스트 데이터의 document 칼럼에 수행

```
from sklearn.metrics import accuracy_score

# 학습 데이터를 적용한 TfidfVectorizer를 이용하여 테스트 데이터를 TF-IDF 값으로 Feature 변환함.
tfidf_matrix_test = tfidf_vect.transform(test_df['document'])

# classifier 는 GridSearchCV에서 최적 파라미터로 학습된 classifier를 그대로 이용
best_estimator = grid_cv.best_estimator_
preds = best_estimator.predict(tfidf_matrix_test)

print('Logistic Regression 정확도: ',accuracy_score(test_df['label'],preds))

Logistic Regression 정확도: 0.86172
```

